Homework 4 – Entropy Coding

314552046 莊明憲

SPEC

- Run length encoding and run length decoding
  - 8x8 block-based DCT coefficients of "lena.png."
  - Quantize the coefficients with the two quantization tables.
  - Use a raster scan to visit all 8x8 blocks in these images.
  - Do the run length encoding by using a zigzag scan to visit all pixels in one block.
  - Do the run length decoding and IDCT to recover the image.
  - Compare the encoded image sizes with the two quantization tables.

10	7	6	10	14	24	31	37
7	7	8	11	16	35	36	33
8	8	10	14	24	34	41	34
8	10	13	17	31	52	48	37
11	13	22	34	41	65	62	46
14	21	33	38	49	62	68	55
29	38	47	52	62	73	72	61
43	55	57	59	67	60	62	59

Quantization table 1

10	11	14	28	59	59	59	59
11	13	16	40	59	59	59	59
14	16	34	59	59	59	59	59
28	40	59	59	59	59	59	59
59	59	59	59	59	59	59	59
59	59	59	59	59	59	59	59
59	59	59	59	59	59	59	59
59	59	59	59	59	59	59	59

Quantization table 2

Ref: https://www.youtube.com/watch?v=Q2aEzeMDHMA

Ref: https://q-viper.github.io/2021/05/24/coding-run-length-encoding-in-python/

Code Explanation

- **Zigzag Scan / Inverse Zigzag**

These two functions convert an 8×8 block into a 1×64 sequence and reconstruct the block using the predefined zigzag index map. They ensure low-frequency coefficients appear first, which improves RLE efficiency.

```python
def zigzag_scan(block):
    """
    Convert 8x8 block to 1x64 vector using zigzag order.
    """
    return block.flatten()[ZIGZAG_INDICES]

def inverse_zigzag_scan(vector):
    """
    Convert 1x64 vector back to 8x8 block.
    """
    if len(vector) != 64:
        raise ValueError("Vector length must be 64 for inverse zigzag scan")
    return vector.flatten()[INVERSE_ZIGZAG_INDICES].reshape((8, 8))
```

- ## Quantization Step

During encoding, each DCT block is divided by the selected quantization table and rounded. This step introduces controlled information loss and directly affects both compression level and reconstruction quality.

```python
quantized_block = np.round(dct_block / q_table).astype(int)
```

- ## Run-Length Encoding (RLE)

The RLE function scans the zigzag vector and converts consecutive zeros into (zero_count, value) pairs. An (0, 0) symbol is appended at the end of each block to mark EOB.

```python
def run_length_encoding(vector):
    """
    Apply Run-Length Encoding (RLE) to the 1D zigzag vector.
    """
    encoded_data = []
    zero_count = 0
    for val in vector:
        if val == 0:
            zero_count += 1
        else:
            encoded_data.append((zero_count, int(val)))
            zero_count = 0
    encoded_data.append((0, 0)) # EOB marker
    return encoded_data
```

- ## Run-Length Decoding

The decoder reconstructs the 64-length vector by expanding zeros according to zero_count and inserting values. It stops when the EOB marker appears.

```python
def run_length_decoding(rle_data):
    """
    Decode RLE data back into a 1x64 vector.
    """
    vector = np.zeros(64, dtype=int)
    index = 0
    for zero_count, value in rle_data:
        if (zero_count, value) == (0, 0): # EOB
            break
        index += zero_count
        if index < 64:
            vector[index] = value
            index += 1
        else:
            break
    return vector
```

- **Block Encoding Pipeline (encode_image)**

For each 8×8 block, the code performs DCT → quantization → zigzag → RLE and stores the results. It also counts output RLE symbols to estimate the compressed size.

```python
def encode_image(image_array, q_table):
    """
    Run the full encoding pipeline on a grayscale image array.
    """
    height, width = image_array.shape
    encoded_blocks = []
    for y in range(0, height, 8):
        for x in range(0, width, 8):
            # 1. Extract 8x8 block
            block = image_array[y:y+8, x:x+8]
            # 2. Level Shift: [0, 255] -> [-128, 127]
            block = block.astype(np.float32) - 128.0
            # 3. Apply 2D DCT
            dct_block = dct2(block)
            # 4. Quantize
            quantized_block = np.round(dct_block / q_table).astype(int)
            # 5. Zigzag Scan
            zigzag_vector = zigzag_scan(quantized_block)
            # 6. Run-Length Encoding
            rle_data = run_length_encoding(zigzag_vector)

            encoded_blocks.append(rle_data)
    return encoded_blocks
```

- **Block Decoding Pipeline (decode_image)**

The decoder reverses the entire process: RLE → inverse zigzag → dequantization → IDCT. Recovered blocks are placed back into the output image.

```python
def decode_image(encoded_blocks, image_shape, q_table):
    """
    Run the full decoding pipeline to recover the image.
    """
    height, width = image_shape
    recovered_image_array = np.zeros(image_shape, dtype=np.float32)
    block_index = 0
    for y in range(0, height, 8):
        for x in range(0, width, 8):
            if block_index >= len(encoded_blocks):
                break

            rle_data = encoded_blocks[block_index]

            # 1. Run-Length Decoding
            zigzag_vector = run_length_decoding(rle_data)
            # 2. Inverse Zigzag Scan
            quantized_block = inverse_zigzag_scan(zigzag_vector)
            # 3. De-quantization
            dequantized_block = quantized_block.astype(np.float32) * q_table
            # 4. Apply 2D Inverse DCT
            idct_block = idct2(dequantized_block)
            # 5. Reverse Level Shift: [-128, 127] -> [0, 255]
            recovered_block = idct_block + 128.0
            # 6. Place the 8x8 block back into the image
            recovered_image_array[y:y+8, x:x+8] = recovered_block

            block_index += 1

    recovered_image_array = np.clip(recovered_image_array, 0, 255)
    return recovered_image_array.astype(np.uint8)
```

Comparison

Original (Cropped)

Reconstructed (Q1)

Reconstructed (Q2)

Results

```
PS E:\Documents\課程內容\碩一上\視訊壓縮\HW4 - Entropy Coding> python VC_HW4_314552046.py
Processing with Quantization Table 1
============================================================
Original image shape: (512, 512)
Processed (cropped) shape: (512, 512)
Original size: 262144 bytes
Encoded size (estimated): 87632 bytes (43816 RLE symbols)
Compression ratio (estimated): 2.99 : 1

Processing with Quantization Table 2
============================================================
Original image shape: (512, 512)
Processed (cropped) shape: (512, 512)
Original size: 262144 bytes
Encoded size (estimated): 52864 bytes (26432 RLE symbols)
Compression ratio (estimated): 4.96 : 1

Decoding images
============================================================
Saved: output_images\lena_recovered_table1.png
Saved: output_images\lena_recovered_table2.png
Comparison image saved: output_images\comparison.png

Image Quality Metrics:
============================================================
PSNR with Q1: 37.24 dB
PSNR with Q2: 34.34 dB

Analysis:
============================================================
Q1 uses finer quantization -> Better quality (Higher PSNR), larger size (Higher 2-Byte RLE symbols)
Q2 uses coarser quantization -> Lower quality (Lower PSNR), smaller size (Fewer 2-Byte RLE symbols)
============================================================
```

Analysis

Using Q1 results in higher-quality reconstruction and a larger encoded size because the quantization values are finer and fewer coefficients become zero. In contrast, Q2 uses stronger quantization, producing more zeros and shorter RLE runs, which leads to a smaller encoded size but lower PSNR. The results clearly show the expected trade-off: Q1 gives better visual quality, while Q2 provides better compression efficiency.